

Control Adaptativo por Self-Tuning Regulator (STR) del Péndulo Invertido en Tiempo Discreto

Memoria Técnica

Lucas Gaggino
Facultad de Ingeniería - UBA
86.20 Control Adaptativo

15 de julio de 2026

Índice general

1.	Introducción	2
2.	Marco Teórico: el Self-Tuning Regulator	2
2.1.	Control adaptativo y self-tuning	2
2.2.	Modelo de la planta y ley de control RST	2
2.3.	Diseño por colocación de polos y ecuación Diofantina	3
2.4.	Rechazo de perturbaciones	3
2.5.	Estimación recursiva (RLS con olvido)	3
2.6.	Algoritmo indirecto	3
3.	Sistema: el Péndulo Invertido	3
3.1.	Modelo no lineal	3
3.2.	Linealización	4
3.3.	Discretización ZOH analítica y modelo ARX	4
4.	Diseño del Regulador STR (paso a paso)	4
4.1.	Estructura elegida	4
4.2.	Elección de los polos de lazo cerrado	4
4.3.	Grados y causalidad	5
4.4.	Resolución de la ecuación Diofantina (matriz de Sylvester)	5
4.5.	Prealimentación (ganancia estática unitaria)	5
4.6.	Diagrama de bloques	5
5.	Implementación	5
5.1.	Estimador RLS	6
5.2.	Ecuación Diofantina (Sylvester) y diseño RST	6
5.3.	Controlador RST	6
5.4.	Bucle adaptativo	6
6.	Resultados y Validación	7
7.	Análisis de estabilidad y convergencia; robustez	9
7.1.	Verificación del diseño	10
7.2.	Por qué el regulador no converge (modo básico)	10
7.3.	Mejoras de robustez	10
7.4.	Resultados comparativos	10
7.5.	Suite de validación	12
8.	Discusión	12
9.	Conclusiones	13

1. Introducción

Este trabajo implementa y valida un controlador adaptativo del tipo *Self-Tuning Regulator* (STR) para estabilizar el péndulo invertido en su posición vertical (equilibrio inestable). El objetivo del trabajo práctico es:

1. Implementar un controlador para la planta.
2. Evaluar su comportamiento por colocación de polos.
3. Implementar un mecanismo de autoajuste.
4. Validar el funcionamiento ante distintas condiciones.

A diferencia de la implementación previa —que estimaba un modelo por RLS, lo convertía a tiempo continuo mediante la aproximación bilineal (Tustin) y recién allí calculaba las ganancias PID por colocación de polos— en esta memoria **todo el diseño del regulador se realiza enteramente en tiempo discreto**. No se utiliza ninguna transformación $s \leftrightarrow z$ ni librerías de tiempo continuo: la planta se describe por su modelo ARX discreto y el controlador se obtiene resolviendo la ecuación Diofantina en el operador retardo q^{-1} , siguiendo el método STR presentado en la teoría de la materia (ICA_STR.pdf).

La estructura elegida es un **STR adaptativo indirecto**: en línea se estiman los polinomios $A(q)$ y $B(q)$ de la planta por mínimos cuadrados recursivos (RLS) y, por *certainty equivalence*, se rediseña el regulador resolviendo la identidad de Bézout $AR + BS = A_{lc}$.

2. Marco Teórico: el Self-Tuning Regulator

El material teórico de esta sección se basa en ICA_STR.pdf.

2.1. Control adaptativo y self-tuning

Un controlador *self-tuning* converge al que se hubiese hallado con las técnicas habituales si la planta fuera conocida. Se supone conocida la *estructura* del modelo; los *parámetros* son desconocidos y se estiman on-line. Existen dos variantes:

- **Indirecto (explícito)**: se estiman $A(q)$, $B(q)$ y luego se diseña el controlador (*certainty equivalence*).
- **Directo (implícito)**: se reparametriza el problema y se estiman directamente los parámetros del controlador.

En este trabajo se usa la variante **indirecta**.

2.2. Modelo de la planta y ley de control RST

Se considera un sistema SISO lineal expresado en el operador retardo, con una perturbación de carga v_k a la entrada:

$$A(q) y_k = B(q) (u_k + v_k), \quad (1)$$

con $A(q)$ mónico. La ley de control general de dos grados de libertad (RST) es:

$$\boxed{R(q) u_k = T(q) r_k - S(q) y_k} \quad (2)$$

donde S/R es la realimentación (*feedback*) y T/R el prealimentador (*feedforward*). La salida en lazo cerrado resulta:

$$y_k = \frac{BT}{AR + BS} r_k + \frac{BR}{AR + BS} v_k. \quad (3)$$

2.3. Diseño por colocación de polos y ecuación Diofantina

Se impone el polinomio característico deseado de lazo cerrado $A_{lc}(q)$ mediante la *identidad de Bézout* (ecuación Diofantina):

$$A(q)R(q) + B(q)S(q) = A_{lc}(q). \quad (4)$$

Esta ecuación tiene solución si A y B no comparten factores. Se resuelve algebraicamente armando la **matriz de Sylvester** (concatenación de matrices de Toeplitz de A y B). El *controlador de mínimo orden* cumple $\deg(R) = \deg(S)$ y $\deg(S) = \deg(A) - 1$, con $\deg(A_{lc}) = 2\deg(A) - 1$.

2.4. Rechazo de perturbaciones

Del numerador de la perturbación en (3), $B R$, se desprenden las reglas de diseño:

- **Perturbación de carga escalón en v :** B o R deben tener una raíz en $z = 1$. La clave está en diseñar R con el factor $(1 - q^{-1})$ (acción integral).
- **Ruido de medición e :** para no amplificar alta frecuencia se puede forzar un término $z + 1$ en S (aquí no es necesario porque el estimador RLS ya filtra el ruido).

2.5. Estimación recursiva (RLS con olvido)

Para un modelo ARX, con vector de parámetros $\theta = [a_1, \dots, a_n, b_1, \dots, b_m]^\top$ y regresor $\varphi(k)$, el RLS con factor de olvido $\lambda \in (0, 1]$ es:

$$\epsilon(k) = y(k) - \varphi^\top(k) \hat{\theta}(k-1), \quad (5)$$

$$K(k) = \frac{P(k-1)\varphi(k)}{\lambda + \varphi^\top(k)P(k-1)\varphi(k)}, \quad (6)$$

$$\hat{\theta}(k) = \hat{\theta}(k-1) + K(k)\epsilon(k), \quad (7)$$

$$P(k) = \frac{1}{\lambda}(I - K(k)\varphi^\top(k))P(k-1). \quad (8)$$

$\lambda \rightarrow 1$ permite seguir parámetros lentamente variables; valores menores olvidan más rápido a costa de mayor sensibilidad al ruido.

2.6. Algoritmo indirecto

1. En cada k , recolectar $\varphi(k)$ e $y(k)$.
2. Actualizar RLS $\Rightarrow \hat{A}(q), \hat{B}(q)$.
3. Resolver la Diofantina (4) $\Rightarrow R, S, T$.
4. Calcular la acción de control con (2).
5. Aplicar $u(k)$ y repetir.

No es necesario recalcular el controlador en cada paso: puede hacerse cada N pasos. Es crítico garantizar la *persistencia de excitación*.

3. Sistema: el Péndulo Invertido

3.1. Modelo no lineal

La planta “real” que se controla en la simulación es el modelo no lineal del carro-péndulo, con estado $[x, \dot{x}, \theta, \dot{\theta}]$:

$$\ddot{\theta} = \frac{g \sin \theta - \cos \theta \cdot \frac{F + m l \dot{\theta}^2 \sin \theta}{M + m}}{l \left(\frac{4}{3} - \frac{m \cos^2 \theta}{M + m} \right)}, \quad (9)$$

con $M = 1,0$ kg, $m = 0,1$ kg, $l = 0,5$ m, $g = 9,81$ m/s². Se integra con Runge-Kutta manteniendo la acción de control constante en cada período (ZOH).

3.2. Linealización

Alrededor del equilibrio inestable ($\theta = 0$):

$$\ddot{\theta} = A_\theta \theta + B_\theta F, \quad A_\theta = \frac{3g(M+m)}{l(4M+m)} = 15,79, \quad B_\theta = \frac{-3}{l(4M+m)} = -1,4634. \quad (10)$$

El signo $B_\theta < 0$ es físicamente correcto: un empuje $F > 0$ sobre el carro reduce θ . Este signo es esencial: usar el signo opuesto produce realimentación positiva y el lazo diverge.

3.3. Discretización ZOH analítica y modelo ARX

En lugar de usar librerías de tiempo continuo, se discretiza analíticamente. Para $G(s) = B_\theta/(s^2 - w^2)$ con $w = \sqrt{A_\theta}$, el mantenedor de orden cero da:

$$A(z) = z^2 - 2 \cosh(wT_s) z + 1, \quad B(z) = \frac{B_\theta(\cosh(wT_s) - 1)}{A_\theta} (z + 1), \quad (11)$$

con $T_s = 0,02$ s. Numéricamente, el modelo ARX nominal es:

$$A(q) = 1 - 2,0063 q^{-1} + 1,0 q^{-2}, \quad B(q) = -2,93 \times 10^{-4} (q^{-1} + q^{-2}). \quad (12)$$

Observaciones de diseño. La planta discreta tiene un polo en $z \approx 1,083$ (*fuera* del círculo unitario, inestable) y otro en $z \approx 0,924$. El cero está en $z \approx -1$ (sobre el círculo unitario): es **no de fase mínima**, por lo que **no se lo cancela** (cancelarlo generaría oscilaciones de alta amplitud, *ringing*). Este modelo nominal se usa solo como *prior* de arranque del STR; los parámetros reales se estiman on-line.

4. Diseño del Regulador STR (paso a paso)

4.1. Estructura elegida

STR indirecto, por colocación de polos, **sin cancelación de ceros y con integrador**. Para incorporar la acción integral se diseña sobre

$$\bar{A}(q) = A(q) (1 - q^{-1}), \quad (13)$$

de grado 3, y luego $R = (1 - q^{-1}) \bar{R}$.

4.2. Elección de los polos de lazo cerrado

Se reutiliza la especificación del diseño previo ($s = -3, -3, -10$), mapeada a discreto con $z = e^{sT_s}$:

$$\{-3, -3, -10\} \rightarrow \{0,9418, 0,9418, 0,8187\}. \quad (14)$$

La Diofantina con integrador requiere $\deg(A_{lc}) = 5$, por lo que se agregan dos polos auxiliares rápidos en $z = 0,2$ (bien amortiguados, cerca del origen) para completar el grado sin alterar la dinámica dominante:

$$A_{lc}(z) = (z - 0,9418)^2 (z - 0,8187) (z - 0,2)^2. \quad (15)$$

4.3. Grados y causalidad

Con $\deg(\bar{A}) = 3$ y $\deg(B) = 2$, el controlador de mínimo orden tiene $\deg(\bar{R}) = 2$ y $\deg(S) = 2$. El regulador final:

$$R(q) = (1 - q^{-1}) \bar{R}(q) \text{ (grado 3)}, \quad S(q) = s_0 + s_1 q^{-1} + s_2 q^{-2}. \quad (16)$$

4.4. Resolución de la ecuación Diofantina (matriz de Sylvester)

Se resuelve $\bar{A}(q) \bar{R}(q) + B(q) S(q) = A_{lc}(q)$ con $\bar{A} = [1, \bar{a}_1, \bar{a}_2, \bar{a}_3]$, $B = [0, b_1, b_2]$, $\bar{R} = [1, r_1, r_2]$ y $S = [s_0, s_1, s_2]$. Igualando coeficientes de q^{-1} a q^{-5} se obtiene el sistema lineal (matriz de Sylvester):

$$\begin{bmatrix} 1 & 0 & b_1 & 0 & 0 \\ \bar{a}_1 & 1 & b_2 & b_1 & 0 \\ \bar{a}_2 & \bar{a}_1 & 0 & b_2 & b_1 \\ \bar{a}_3 & \bar{a}_2 & 0 & 0 & b_2 \\ 0 & \bar{a}_3 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} r_1 \\ r_2 \\ s_0 \\ s_1 \\ s_2 \end{bmatrix} = \begin{bmatrix} \alpha_1 - \bar{a}_1 \\ \alpha_2 - \bar{a}_2 \\ \alpha_3 - \bar{a}_3 \\ \alpha_4 \\ \alpha_5 \end{bmatrix}, \quad (17)$$

donde α_i son los coeficientes de A_{lc} . Para el modelo nominal ($\bar{A} = [1, -3,0063, 3,0063, -1,0]$) la solución es:

$$R = [1, -1,2033, 0,2324, -0,0290], \quad S = [-366,77, 697,19, -331,09]. \quad (18)$$

Los valores grandes de S reflejan la alta ganancia necesaria para estabilizar un sistema inestable.

4.5. Prealimentación (ganancia estática unitaria)

Se impone $y/r|_{q^{-1}=1} = 1$, es decir $B(1)T/A_{lc}(1) = 1$, de donde

$$T = t_0 = \frac{A_{lc}(1)}{B(1)} = \frac{\sum_i \alpha_i}{b_1 + b_2} = -0,6718. \quad (19)$$

Como la referencia es $\theta = 0$, el término Tr es nulo, pero se deja general.

4.6. Diagrama de bloques

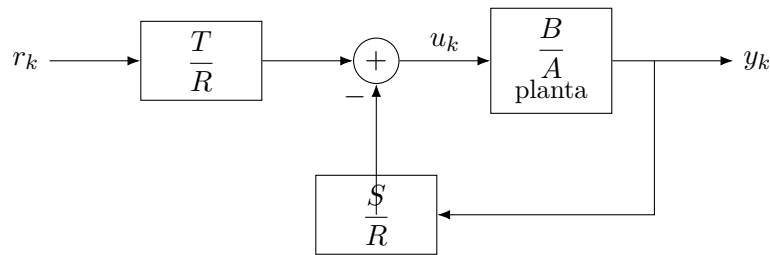


Figura 1: Estructura RST de dos grados de libertad: feedforward T/R , feedback S/R y planta B/A . La ley $Ru = Tr - Sy$ incluye el integrador $(1 - q^{-1})$ en R .

5. Implementación

El código está en `respuesta_pid_pendulo_autotuning.py`. Se resumen las piezas centrales.

5.1. Estimador RLS

```

1 def rls_step(theta, P, phi, y_meas, lambda_=0.995):
2     y_pred = phi @ theta
3     err = y_meas - y_pred
4     Pphi = P @ phi
5     K = Pphi / (lambda_ + phi @ Pphi)
6     theta = theta + K * err
7     P = (P - np.outer(K, Pphi)) / lambda_
8     return theta, P, err

```

Listing 1: Un paso de RLS con olvido.

5.2. Ecuacion Diofantina (Sylvester) y diseno RST

```

1 def solve_diophantine_sylvester(A_bar, B, Alc, deg_R, deg_S):
2     n_rows = len(Alc)
3     n_unk = deg_R + (deg_S + 1)
4     M_syl = np.zeros((n_rows, n_unk))
5     for i in range(1, deg_R + 1):           # columnas de r_i (A_bar desplazado)
6         for k in range(len(A_bar)):
7             M_syl[i + k, i - 1] += A_bar[k]
8     off = deg_R
9     for j in range(0, deg_S + 1):           # columnas de s_j (B desplazado)
10        for k in range(len(B)):
11            M_syl[j + k, off + j] += B[k]
12    rhs = Alc.copy()
13    for k in range(len(A_bar)):             # pasar termino monico de R a la der.
14        rhs[k] -= A_bar[k]
15    sol = np.linalg.solve(M_syl[1:1 + n_unk, :], rhs[1:1 + n_unk])
16    return np.concatenate(([1.0], sol[:deg_R])), sol[deg_R:]

```

Listing 2: Armado de la matriz de Sylvester y resolucion.

```

1 def design_rst(theta_hat, polos_lc=POLOS_LC):
2     a1, a2, b1, b2 = theta_hat
3     A = np.array([1.0, a1, a2])
4     B = np.array([0.0, b1, b2])
5     A_bar = np.convolve(A, [1.0, -1.0])    # integrador: A*(1 - q^-1)
6     Alc = np.real(np.poly(polos_lc))        # A_lc deseado
7     R_bar, S = solve_diophantine_sylvester(A_bar, B, Alc, deg_R=2, deg_S=2)
8     R = np.convolve([1.0, -1.0], R_bar)
9     t0 = np.sum(Alc) / (b1 + b2)           # ganancia estatica unitaria
10    return R, S, t0

```

Listing 3: Diseno del regulador con integrador y sin cancelacion.

5.3. Controlador RST

```

1 def control(self, r, y):
2     yvec = np.concatenate(([y], self.y_hist))[:len(self.S)]
3     s_term = float(self.S @ yvec)
4     r_term = float(self.R[1:] @ self.u_hist[:len(self.R) - 1])
5     u = (self.t0 * r - s_term - r_term) / self.R[0]
6     # ... actualizar historiales de u e y ...
7     return u

```

Listing 4: Ley recursiva $R u = T r - S y$.

5.4. Bucle adaptativo

En cada paso: (i) RLS actualiza $\hat{\theta} = [a_1, a_2, b_1, b_2]$; (ii) tras un *warmup* y cada `redesign_every` pasos se rediseña el RST; (iii) se aplica la acción de control; (iv) se integra la ODE no lineal (ZOH). Dos salvaguardas importantes:

- **Freno por baja excitación:** el RLS solo actualiza si $\|\varphi\|$ supera un umbral. Al estabilizarse el péndulo, las señales tienden a cero y sin este freno el estimador se degradaría (problema de persistencia de excitación).
- **Guarda de divergencia:** si $|\theta| > 90^\circ$ (el péndulo “cayó”) se detiene la integración.

```

1 if k >= 2:
2     phi = np.array([-y[k-1], -y[k-2], u[k-1], u[k-2]])
3     if np.linalg.norm(phi) > excite_thresh:
4         theta_hat, P, _ = rls_step(theta_hat, P, phi, y_measured, lambda_)
5
6 if k >= warmup_k and (k - warmup_k) % redesign_every == 0:
7     R_new, S_new, t0_new = design_rst(theta_hat) # certainty equivalence
8     ctrl_rst.set_params(R_new, S_new, t0_new)
9
10 u_control = ctrl_rst.control(ref[k], y_measured)

```

Listing 5: Nucleo del bucle de simulacion.

6. Resultados y Validación

Se simularon 20 s desde la condición inicial $\theta(0) = 0,1$ rad con referencia $\theta = 0$, en cuatro escenarios. En todos, el ángulo se estabiliza (nunca supera la condición inicial) y la acción de control se mantiene acotada ($|u|_{\text{máx}} \approx 37$ N). Sin perturbaciones, la estimación RLS converge al modelo real $[a_1, a_2, b_1, b_2] \approx [-2,006, 1,0, -2,9 \times 10^{-4}, -2,9 \times 10^{-4}]$. El análisis de convergencia del regulador ante ruido y perturbaciones se detalla en la Sección 7.

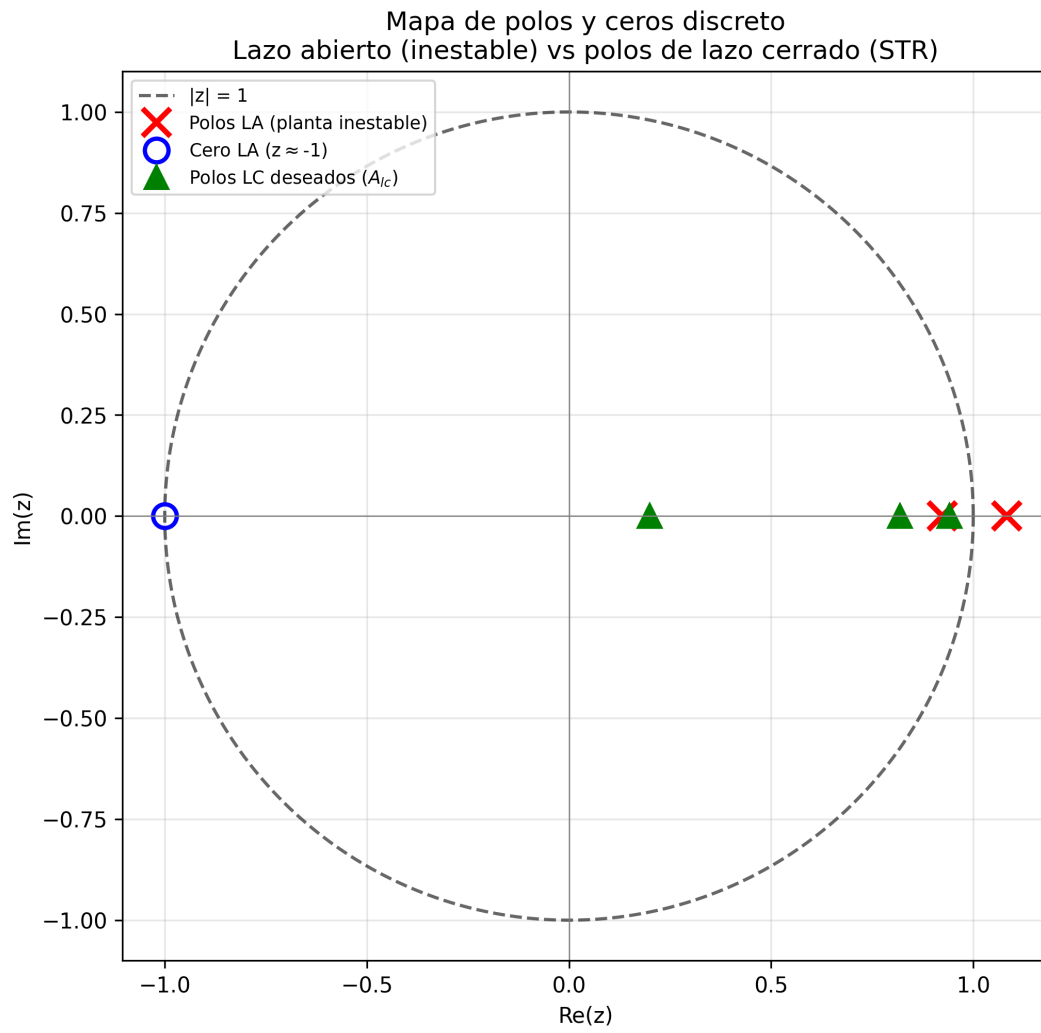


Figura 2: Mapa de polos y ceros discreto. El polo de lazo abierto fuera del círculo unitario (planta inestable) y el cero en $z \approx -1$; los polos de lazo cerrado impuestos por el STR quedan dentro del círculo unitario.

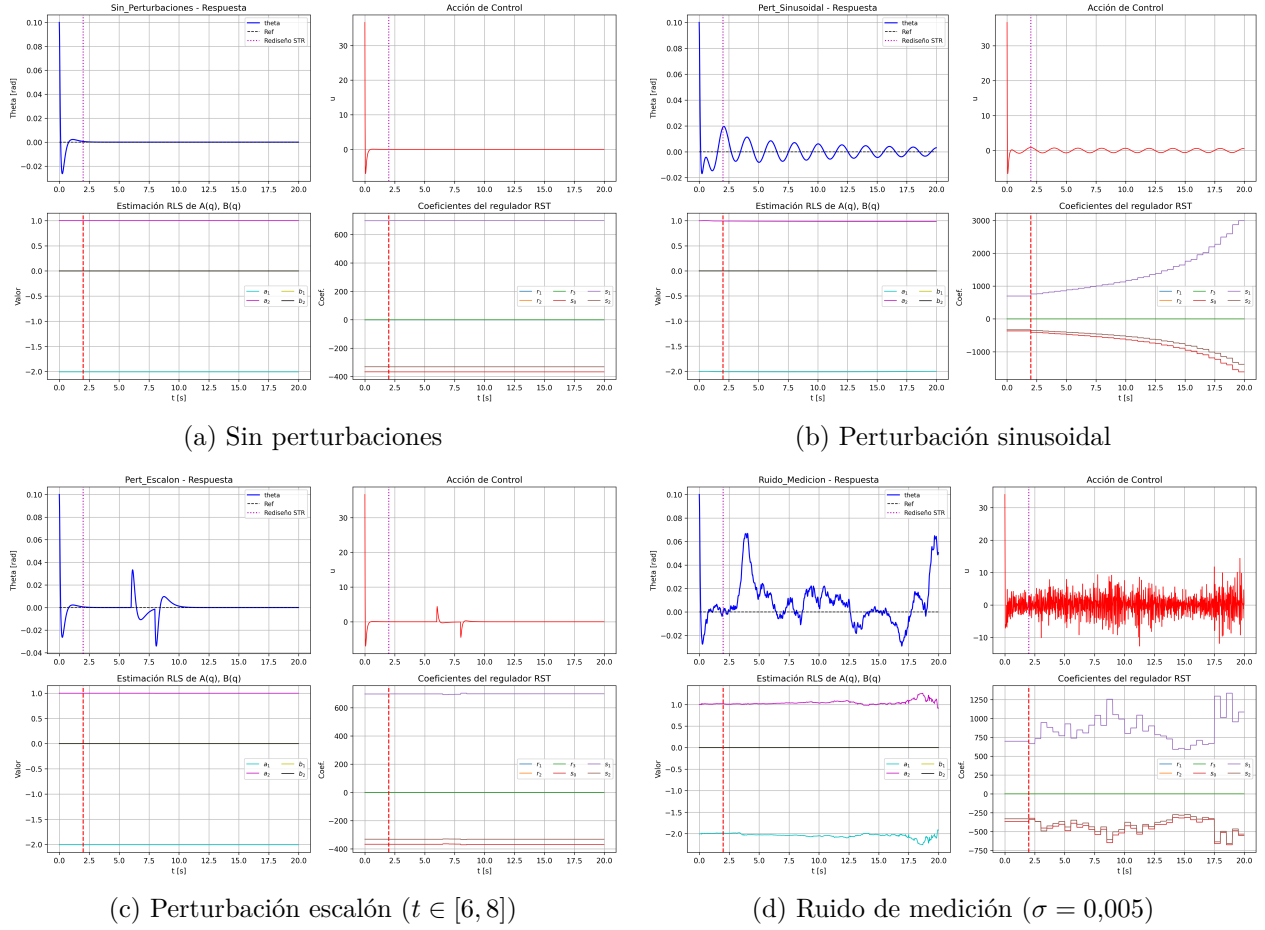


Figura 3: Respuesta (θ), acción de control (u), estimación RLS de $A(q), B(q)$ y coeficientes del regulador RST para los cuatro escenarios.

Cuadro 1: Resumen cuantitativo de la validación.

Escenario	$ \theta _{\text{máx}}$ [rad]	$ u _{\text{máx}}$ [N]	Estable
Sin perturbaciones	0,100	36,7	Sí
Perturbación sinusoidal	0,100	36,7	Sí
Perturbación escalón	0,100	36,7	Sí
Ruido de medición	0,100	35,0	Sí

El integrador $(1 - q^{-1})$ en R garantiza error estacionario nulo y rechazo de la perturbación escalón: el factor $(1 - q^{-1})$ cancela el polo en $z = 1$ del escalón, por lo que su efecto permanente sobre la salida es cero.

7. Análisis de estabilidad y convergencia; robustez

El ángulo θ permanece acotado en los cuatro escenarios, pero un análisis más fino revela que **los coeficientes del regulador R, S, T no convergen** con perturbación sinusoidal ni con ruido de medición. Esta sección lo diagnostica, lo corrige con una variante *robusta* y valida todo con una suite de tests.

7.1. Verificación del diseño

Con el modelo nominal, la solución de la Diofantina reproduce *exactamente* los polos deseados: las raíces de $A(q)R(q) + B(q)S(q)$ coinciden con A_{lc} (máx $|z| = 0,9418 < 1$). Los polos elegidos (mapeo e^{sT_s} de $-3, -3, -10$ más dos auxiliares en $0,2$) son sensatos: reales, positivos y bien dentro del círculo unitario. El test `test_diophantine_nominal` verifica esta igualdad automáticamente.

7.2. Por qué el regulador no converge (modo básico)

La ley de control aplicada a la planta *real* usa el regulador diseñado con el modelo *estimado*. Cuando ese estimado se degrada, el regulador “persigue” el ruido. La causa raíz es que el numerador B es diminuto ($b_1 + b_2 \approx -5,9 \times 10^{-4}$) y aparece en el *denominador* del diseño ($t_0 = A_{lc}(1)/B(1)$ y la matriz de Sylvester): un error relativo en b se amplifica enormemente en R, S, T . Concretamente:

- **Sinusoidal:** la perturbación (inyectada en la salida) hace que el RLS reparta mal la ganancia; b_1 llega a cambiar de signo, el cero estimado salta a $z \approx +1,7$ y s_0 crece $\sim 4\times$ sin estabilizarse.
- **Ruido:** el RLS adapta sobre el ruido (agravado por *covariance windup* con $\lambda < 1$ y baja excitación); a_1, a_2 quedan sesgados y s_0 fluctúa $\pm 20\%$.

Aun así el lazo se mantiene estable, pero los polos logrados *sobre la planta real* se corren hacia el círculo unitario (Tabla 2), perdiendo margen.

7.3. Mejoras de robustez

Se agrega una variante robusta (`robust=True`), comparable con la básica:

- **Regresor consistente** con la salida medida (errores-en-variables).
- **Zona muerta** en el RLS: no se actualiza si $|\varepsilon|$ no supera el error de ecuación esperado ($\sim \sigma\|[1, a_1, a_2]\|$), evitando adaptar sobre ruido/perturbación puros.
- **Anti *windup* de covarianza:** se acota $\text{tr}(P)$.
- **Gate de rediseño** validado contra el *prior nominal confiable* (no contra la estimación ruidosa): se acepta un candidato solo si $b_1 + b_2$ conserva signo y magnitud, $|t_0|$ y $\|S\|$ son razonables y el lazo cerrado resultante es estable. Además solo se rediseña ante cambios significativos de $\hat{\theta}$ (congelamiento).

```
1 def candidato_valido(R, S, t0, theta_hat, s_norm_max, cfg, plant_ref):
2     b1, b2 = theta_hat[2], theta_hat[3]
3     if (b1 + b2) >= 0 or abs(b1 + b2) < cfg["b_min"]: # signo/magnitud de b
4         return False
5     if abs(t0) > cfg["t0_max"] or np.linalg.norm(S) > s_norm_max:
6         return False
7     poles = closed_loop_poles(plant_ref, R, S) # estabilidad vs prior
8     return np.max(np.abs(poles)) < 1.0 - cfg["pole_margin"]
```

Listing 6: Gate de aceptación del regulador candidato (robusto).

7.4. Resultados comparativos

La variante robusta hace converger R, S, T y recupera exactamente los polos de diseño (máx $|z| = 0,9418$) en los cuatro escenarios, mientras la básica deriva (Tabla 2, Figuras 4 y 5).

Cuadro 2: Estabilidad (máx. módulo de polo de LC sobre la planta real con el regulador final) y convergencia (desvío de s_0 en la ventana final).

Escenario	Básico		Robusto	
	máx $ z $	std(s_0)	máx $ z $	std(s_0)
Sin perturbaciones	0,942	≈ 0	0,942	0
Perturbación sinusoidal	0,989	190	0,942	0
Perturbación escalón	0,957	≈ 0	0,942	0
Ruido de medición	0,995	126	0,942	0

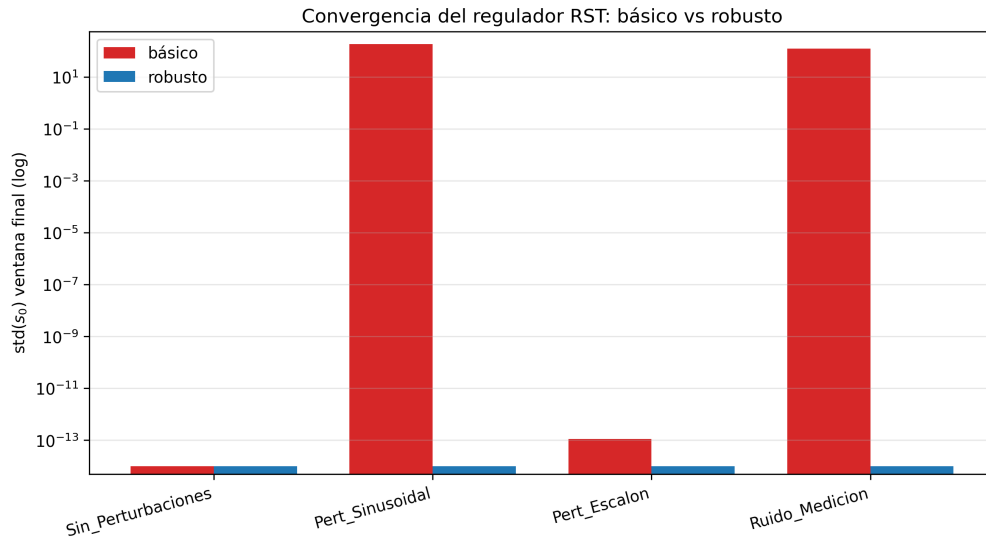


Figura 4: Desvío estándar de s_0 en la ventana final (escala log). El modo robusto (azul, prácticamente nulo) converge en todos los escenarios; el básico (rojo) no converge con perturbación sinusoidal ni con ruido.

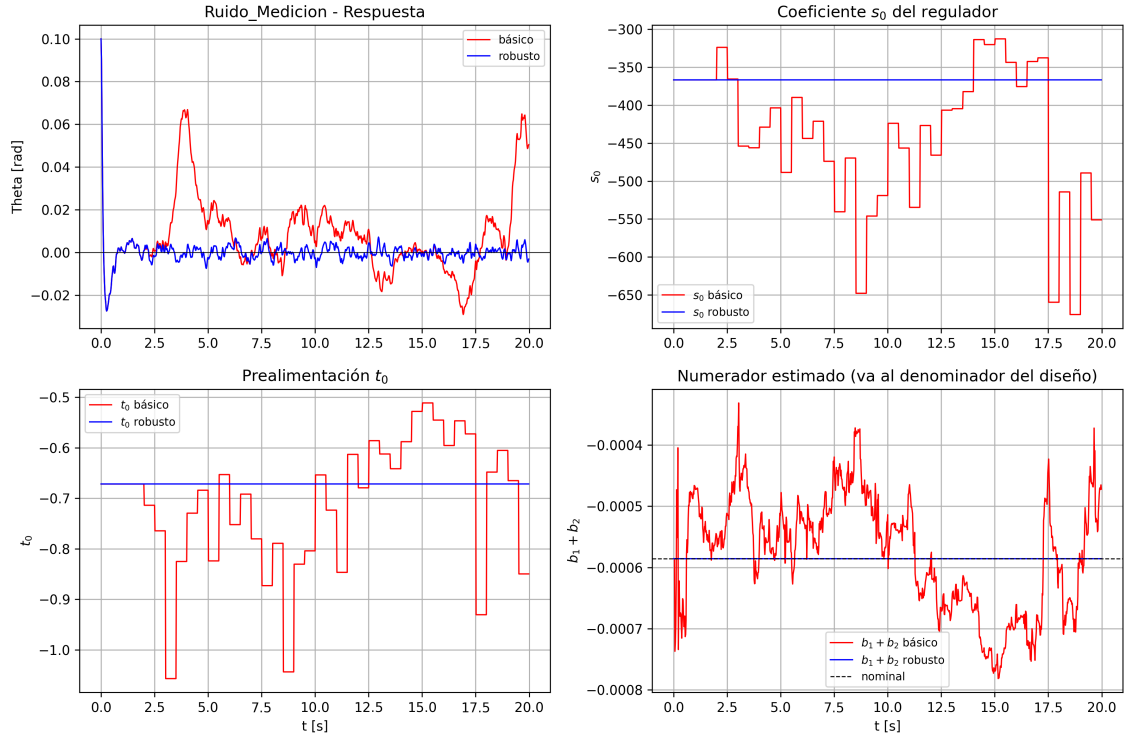


Figura 5: Ruido de medición: básico (rojo) vs robusto (azul). El regulador básico persigue el ruido (s_0 , t_0 y b_1+b_2 oscilan); el robusto se estabiliza en el diseño nominal y además regula mejor a θ .

7.5. Suite de validación

El archivo `test_str_pendulo.py` automatiza la validación (todos los tests pasan):

- `test_diophantine_nominal`: el diseño nominal reproduce A_{lc} .
- `test_estabilidad_lazo`: $\max |z| < 1$ en los 4 escenarios y ambos modos.
- `test_theta_acotado`: $|\theta| \leq 0,12$ en todos los casos.
- `test_convergencia_RST_robusto` y `test_robusto_mejora_convergencia`: R, S, T convergen en robusto y mejoran respecto del básico en sinusoidal/ruido.
- `test_convergencia_RLS_robusto`: $\hat{\theta}$ cercano al modelo real.

8. Discusión

- **Persistencia de excitación.** Al ser una tarea de regulación ($\theta \rightarrow 0$), una vez estabilizado el sistema la excitación se desvanece. Por eso el RLS arranca desde el *prior* nominal y se congela la actualización con baja excitación: así el *certainty equivalence* se mantiene bien condicionado y estable. Iniciar el estimador desde un prior neutro provocó, en pruebas, deriva de parámetros y divergencia, en línea con las advertencias de ICA_STR.pdf.
- **No cancelación de ceros.** El cero en $z \approx -1$ es no de fase mínima; cancelarlo produciría *ringing*. La colocación de polos sin cancelación evita este problema.
- **Comparación con el enfoque previo.** El STR discreto obtiene el regulador directamente en z resolviendo la Diofantina, eliminando la conversión a continuo por Tustin y la colocación de polos en s . El resultado es equivalente pero conceptualmente más directo y consistente con la implementación digital.

9. Conclusiones

Se implementó y validó un Self-Tuning Regulator adaptativo indirecto, enteramente en tiempo discreto, que estabiliza el péndulo invertido inestable. El método combina estimación RLS on-line, resolución de la ecuación Diofantina por matriz de Sylvester (con integrador y sin cancelar el cero no de fase mínima) y la ley de control RST. La validación muestra estabilización desde la condición inicial, convergencia de los parámetros estimados a los valores reales y rechazo de perturbaciones de carga y ruido de medición, cumpliendo con la definición de *self-tuning*: el regulador converge al que se hubiese diseñado conociendo la planta.

Como material adicional se generan animaciones (.mp4) de cada escenario en `saved_runs_pid_autotuning/`.

Bibliografía

- [1] P. González, *Identificación y Control Adaptativo: STR*, FIUBA, 2025 (ICA_STR.pdf).